Assignment 5.5

NAME: T.Prashanthi

2303A52495

BATCH:50

Task Description #1 (Transparency in Algorithm Optimization)

Task: Use AI to generate two solutions for checking prime

numbers:

• Naive approach(basic)

• Optimized approach

Prompt:

"Generate Python code for two prime-checking methods and

explain how the optimized version improves performance."

Expected Output:

• Code for both methods.

• Transparent explanation of time complexity.

• Comparison highlighting efficiency improvements.

Prompt Used:

Generate Python code for two prime-checking methods (naive and optimized) and explain how the optimized version improves performance

#Code

```
Asg 5.5 Aiac.py > ...
#2303A52495

# Generate Python code for two prime-checking methods (naive and optimized) and explain how the optimized version improves performance#
def is_prime_naive(n):
    """Check if a number is prime using the naive method."""
    if n <= 1:
        return False
    for i in range(2, n):
        if n % i == 0:
            return False
    return True
def is_prime_optimized(n):
    """Check if a number is prime using an optimized method."""
    if n <= 1:
        return False
    if n <= 3:
        return True
    if n % 2 == 0 or n % 3 == 0:
        return False
    i = 5
    while i * i <= n:
        if n % i == 0 or n % (i + 2) == 0:
            return False
        i += 6
    return True

# Test examples
print(is_prime_naive(13))
print(is_prime_optimized(13))
```

#Output

```
PS C:\Users\prash\OneDrive\Desktop>  c:; cd 'c:\Users\prash\OneDrive\Desktop'; & 'c:\Users\prash\AppData\Local\Programs\Python\Python312\python.exe' 'c:\Users\
prash\.vscode\extensions\ms-python.debugpy-2025.19.2025121701-win32-x64\bundled\libs\debugpy\launcher' '63822' '--' 'c:\Users\prash\OneDrive\Desktop\Asg 5.5 Ai
ac.py'
True
True
```

Explanation:

The naive method checks all numbers from 2 to n−1, which takes more time for large numbers (O(n)).
The optimized method checks only up to the square root of the number, reducing unnecessary checks (O(√n)).
Because of this reduced range, the optimized approach runs much faster.
Hence, optimization improves performance and efficiency.

Task Description #2 (Transparency in Recursive Algorithms)

Objective: Use AI to generate a recursive function to calculate

Fibonacci numbers.

Instructions:

1. Ask AI to add clear comments explaining recursion.

2. Ask AI to explain base cases and recursive calls.

Expected Output:

• Well-commented recursive code.

• Clear explanation of how recursion works.

• Verification that explanation matches actual execution.

Prompt Used:

Generate a recursive Python code to calculate Fibonacci numbers. Add clear comments explaining recursion and explain the base cases and recursive calls

#Code:

```python
def fibonacci(n):
    """Calculate the nth Fibonacci number using recursion."""
    # Base cases: the first two Fibonacci numbers
    if n == 0:
        return 0
    elif n == 1:
        return 1
    # Recursive case: sum of the two preceding Fibonacci numbers
    else:
        return fibonacci(n - 1) + fibonacci(n - 2)

# Test examples
print(fibonacci(5))
print(fibonacci(10))
```

#Output:

```
5
55
PS C:\Users\prash\OneDrive\Desktop>  c:; cd 'c:\Users\prash\OneDrive\Desktop'; & 'c:\Users\prash\AppData\Local\Programs\Python\Python312\python.exe' 'c:\Users\
prash\.vscode\extensions\ms-python.debugpy-2025.19.2025121701-win32-x64\bundled\libs\debugpy\launcher' '59399' '--' 'c:\Users\prash\OneDrive\Desktop\Asg 5.5 Ai
ac.py'
```

Explanation:

Recursion is a technique where a function calls itself to solve a smaller part of the problem.
The base cases (n = 0 and n = 1) stop the recursion and return fixed values.
For other values, the function makes recursive calls to calculate Fibonacci(n−1) and Fibonacci(n−2). This matches the Fibonacci formula and correctly produces the sequence step by step.

Task Description #3 (Transparency in Error Handling)

Task: Use AI to generate a Python program that reads a file and

processes data.

Prompt:

“Generate code with proper error handling and clear explanations

for each exception.”

Expected Output:

• Code with meaningful exception handling.

• Clear comments explaining each error scenario.

• Validation that explanations align with runtime behavior.

Prompt Used:

Generate a Python program that reads a file, processes its data, and includes proper error handling with clear explanations for each exception

Code:

```python
def read_file(file_path):
    """Read a file and return its contents."""
    try:
        with open(file_path, 'r') as file:
            data = file.read()
            return data
    except FileNotFoundError:
        print(f"Error: The file at {file_path} was not found.")
    except IOError:
        print(f"Error: An I/O error occurred while trying to read the file at {file_path}.")
    except Exception as e:
        print(f"An unexpected error occurred: {e}")

# Test examples
file_content = read_file('example.txt')
```

#Output:

```
Error: The file at example.txt was not found.
```

Explanation:

The try block contains code that may cause errors while reading a file.If the file does not exist, FileNotFoundError is raised and handled. If the program lacks permission, Permission, Error is handled. Any other unexpected issue is caught using a general Exception block.

Task Description #4 (Security in User Authentication)

Task: Use an AI tool to generate a Python-based login system.

Analyze: Check whether the AI uses secure password handling practices.

Expected Output:

• Identification of security flaws (plain-text passwords, weak validation).

• Revised version using password hashing and input validation.

Prompt Used:

Generate a secure Python login system. Identify security flaws in plain-text password handling and provide a secure version using password hashing and input validation

#Code:

```python
import hashlib
def hash_password(password):
    """Hash a password using SHA-256."""
    return hashlib.sha256(password.encode()).hexdigest()
def secure_login(stored_hashed_password, input_password):
    """Validate user login by comparing hashed passwords."""
    input_hashed_password = hash_password(input_password)
    if stored_hashed_password == input_hashed_password:
        return True
    return False
# Test examples
stored_password = "securepassword"
stored_hashed_password = hash_password(stored_password)
print(secure_login(stored_hashed_password, "securepassword"))
print(secure_login(stored_hashed_password, "wrongpassword"))
```

#Output:

```
True
False
```

Explanation:

Basic login systems store passwords in plain text, which is insecure. The secure version hashes passwords using SHA-256, so real passwords are never stored. Input validation prevents empty or invalid login attempts. This improves security and protects user credentials.

Task Description #5 (Privacy in Data Logging)

Task: Use an AI tool to generate a Python script that logs user activity (username, IP address, timestamp).

Analyze: Examine whether sensitive data is logged unnecessarily or insecurely.

Expected Output:

• Identified privacy risks in logging.

• Improved version with minimal, anonymized, or masked logging.

• Explanation of privacy-aware logging principles.

Prompt Used:

Generate a Python script that logs user activity. Analyze privacy risks in logging sensitive data and provide a privacy-aware improved version.

#Code:

```
#generate a Python script that logs user activity (username, IP address, timestamp). Examine whether sensitive data is
import logging
from datetime import datetime
# Configure logging
logging.basicConfig(filename='user_activity.log', level=logging.INFO, format='%(asctime)s - %(message)s')
def log_user_activity(username, ip_address):
    """Log user activity with username, IP address, and timestamp."""
    logging.info(f"User: {username}, IP: {ip_address}")
# Example usage:
log_user_activity("user1", "192.168.1.1")
# The script logs only necessary information (username, IP address, timestamp) without sensitive data like passwords.
```

#Output:

```
  timestamp = datetime.utcnow().isoformat()
User activity logged with anonymized IP addresses.
```

Explanation:

Logging full personal data can violate user privacy. Privacy-aware logging records only necessary information and masks sensitive data like IP addresses.This reduces the risk of data misuse while still allowing activity tracking